

Recurrent Neural Networks (RNNs)

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

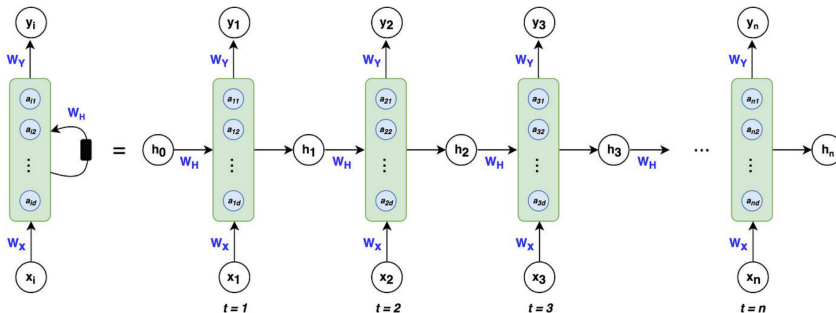
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 31, 2026

1. Motivation
2. Learning Outcomes
3. Introduction
4. RNN Architectures
5. RNN Training Challenges
6. Vanishing and Exploding Gradients
7. LSTM - Long Short-Term Memory
8. GRU - Gated Recurrent Unit
9. LSTM vs GRU
10. Bidirectional RNNs
11. Future Directions
12. Summary
13. References

Motivation: Why Do We Need RNNs?



Source: B. Khuong, "The Basics of Recurrent Neural Networks (RNNs)," Towards AI, 2019

- ▶ Traditional feedforward networks don't handle sequential data effectively.
- ▶ Many applications (language modeling, time-series prediction, speech recognition) require memory of past inputs.
- ▶ RNNs enable temporal dynamic behavior by maintaining hidden states.

Examples Where Order Matters:

- ▶ Translating “I am happy” vs “Happy I am”
- ▶ Predicting next stock price based on past trends
- ▶ Understanding a sentence word-by-word

Key Idea: Add a feedback loop to remember previous computations.

By the end of this session, you should be able to:

- ▶ Understand the structure and working of Recurrent Neural Networks (RNNs).
- ▶ Recognize various RNN architectures (One-to-One, One-to-Many, Many-to-One, Many-to-Many).
- ▶ Explain the concept of shared parameters in RNNs.
- ▶ Describe teacher forcing and the exposure bias it introduces.
- ▶ Explain why RNNs suffer from vanishing and exploding gradients.
- ▶ Understand how Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures solve these problems.
- ▶ Compare LSTM and GRU architectures in terms of performance and complexity.
- ▶ Explain how a bidirectional RNN combines forward and backward context, and why decoders cannot be bidirectional.
- ▶ Recognize limitations and future directions in sequence modeling.

RNNs : Introduction

	Sequence	$P(\text{Sequence})$
 J'ai vu le match de foot 	I saw the game of soccer	4.5 e-5
	I saw the soccer game	<u>6.0 e-5</u>
	I saw the soccer match	4.6 e-5
	Saw I the game of soccer	2.6 e-9

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)

- ▶ Bigrams: $P(w_2|w_1) = \frac{\text{count}(w_1, w_2)}{\text{count}(w_1)}$
- ▶ Trigrams: $P(w_3|w_1, w_2) = \frac{\text{count}(w_1, w_2, w_3)}{\text{count}(w_1, w_2)}$
- ▶ $P(w_1, w_2, w_3) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2)$
- ▶ **Limitations:** Large N-grams need significant space and RAM.

- ▶ N-gram tables consume a lot of memory.
- ▶ **Fixed context window:** only the previous $(n - 1)$ words are conditioned on.
- ▶ **Data sparsity:** larger n implies exponentially many n -grams; many combinations are rare or unseen in training data.
- ▶ **No learning or adaptability:** probabilities come from counting and normalizing, so nothing is shared between words with similar meanings.
- ▶ Different types of **RNNs** are the **preferred alternative**.

A neural network with loops allowing information to persist. **Core Elements:**

- ▶ Hidden State h_t captures memory of previous inputs.
- ▶ Input x_t , Output y_t .
- ▶ Parameter sharing: Same weights used across time steps.

Mathematical Formulation:

- ▶ $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$
- ▶ $y_t = W_{hy}h_t + b_y$

This recurrence allows information to propagate through time.

- ▶ RNNs target **sequential data** where order matters (language, time series, speech).
- ▶ **Capturing context:** a hidden state carries information from previous steps.
- ▶ Example: word meaning often depends on preceding words; RNNs model such dependency.
- ▶ **Variable-length inputs** within a single architecture.
- ▶ **Temporal patterns** in language modeling, speech, financial series, etc.

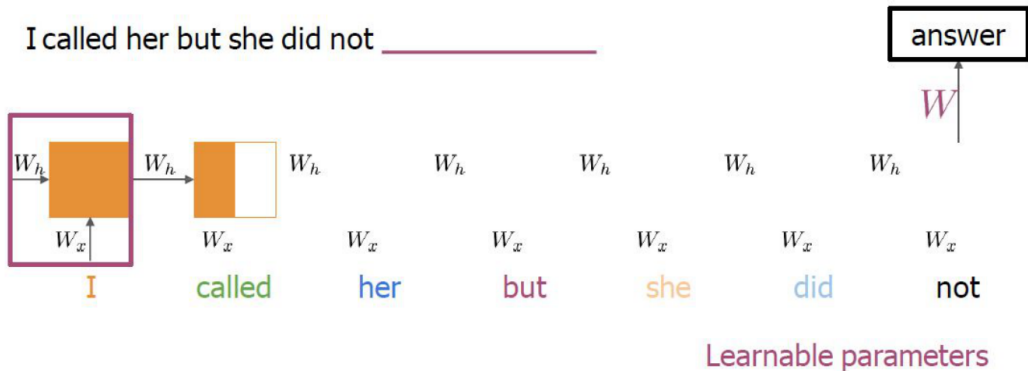
Nour was supposed to study with me. I called her but she **did not** answer

want
respond
choose
want
have
ask
attempt
answer
know

RNNs look at every previous word

Similar probabilities with trigram

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)

RNN : **Architecture**

Unrolled RNN:

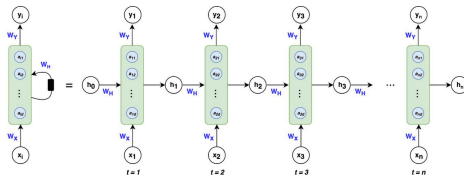
An RNN is essentially a chain of repeating neural network modules, one for each time step.

Each time step shares the same parameters:

- ▶ Input x_t
- ▶ Hidden state h_t
- ▶ Output y_t

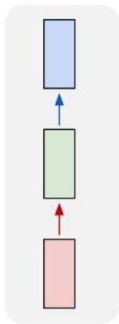
Key Idea

Temporal representation without increasing parameter count!



Source: B. Khuong, "The Basics of Recurrent Neural Networks (RNNs)," Towards AI, 2019

one to one



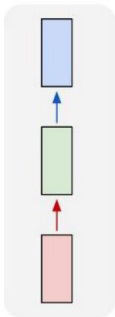
Vanilla Neural Networks

Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks; figure after A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (2015)

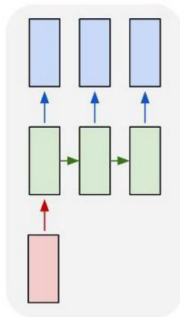


Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)

one to one

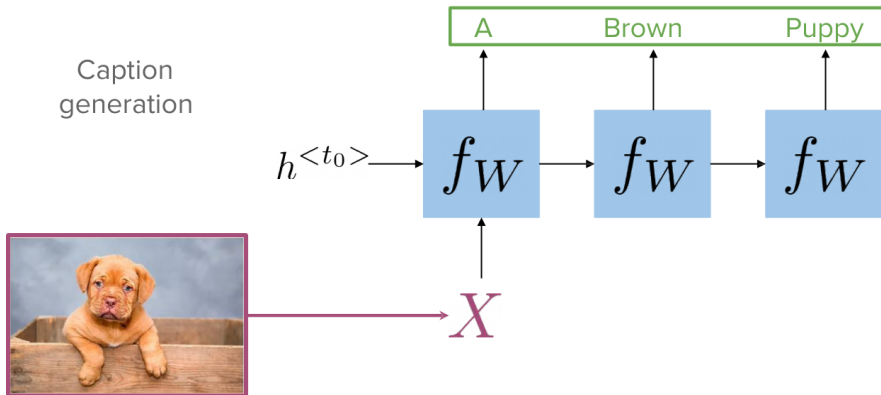


one to many



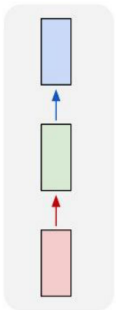
e.g. Image Captioning
image → sequence of words

Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks; figure after A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (2015)

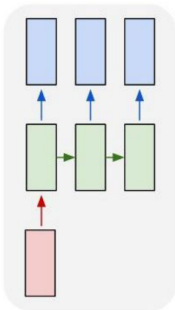


Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)

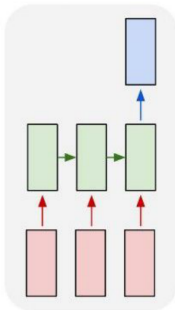
one to one



one to many



many to one



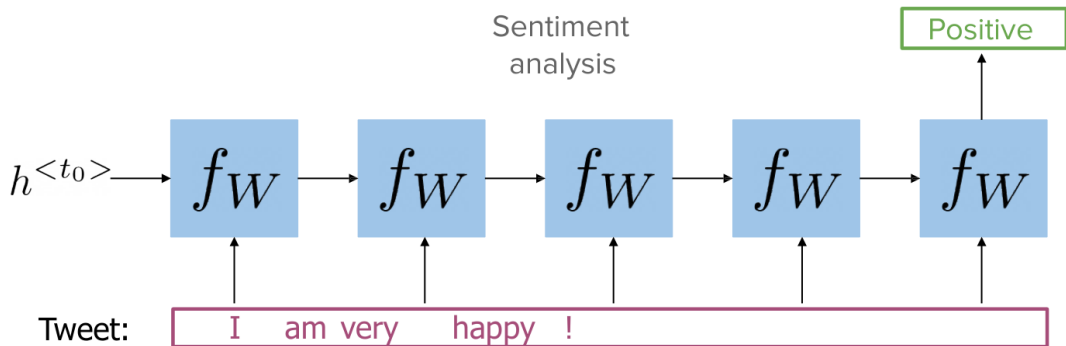
e.g. Sentiment Analysis

sequence of words $\rightarrow$ sentiment

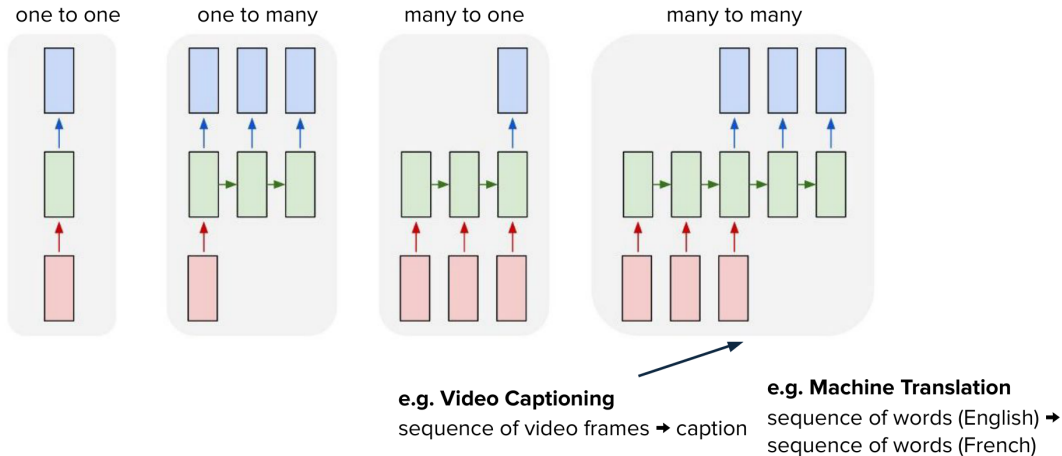
e.g. Action Prediction

sequence of video frames $\rightarrow$ action class

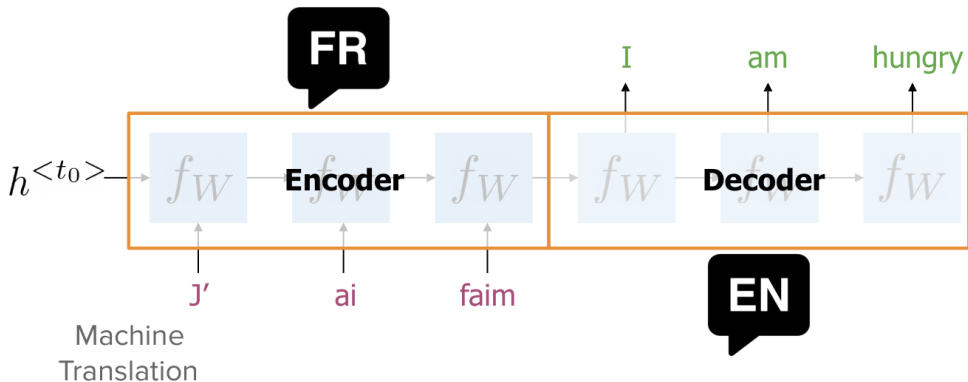
Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks; figure after A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (2015)



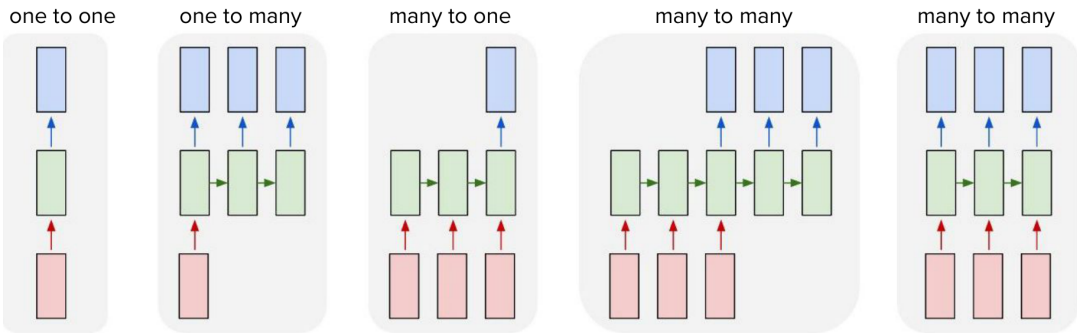
Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks; figure after A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (2015)

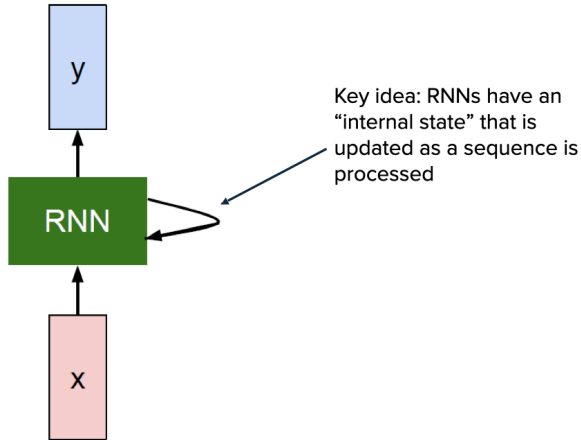


Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



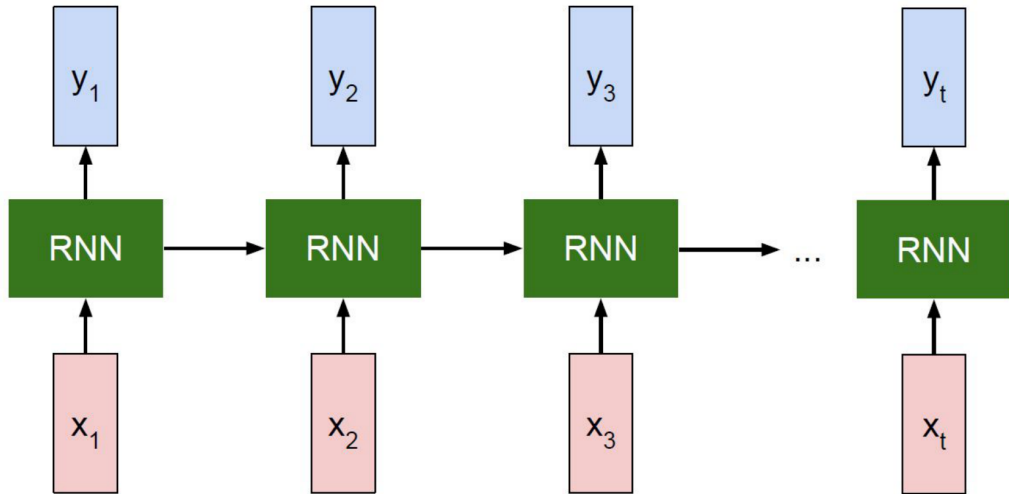
e.g. Video classification on frame level

Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks; figure after A. Karpathy, "The Unreasonable Effectiveness of Recurrent Neural Networks" (2015)



Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

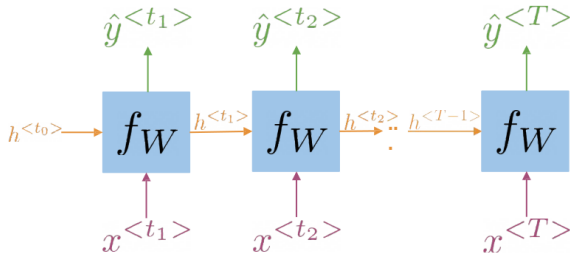
Unrolled RNN



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

RNN : Mathematical Formulations

- ▶ How RNNs propagate information (Through time!)
- ▶ How RNNs make predictions

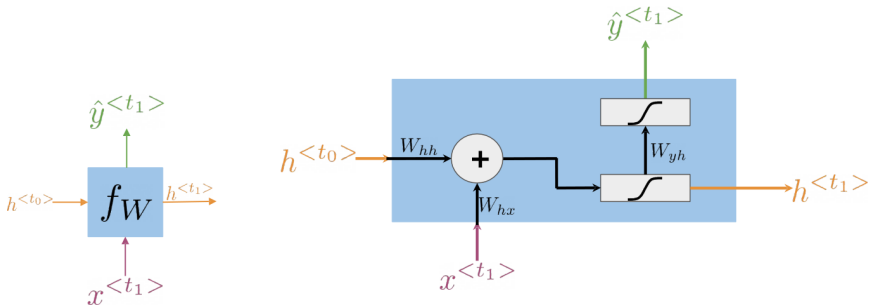


$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

$$h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h)$$

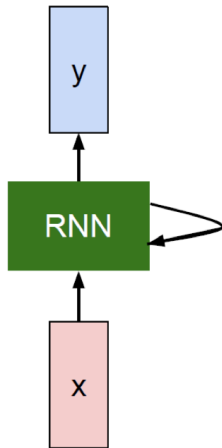
Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



$$h^{<t>} = g(W_{hh}h^{<t-1>} + W_{hx}x^{<t>} + b_h)$$

$$\hat{y}^{<t>} = g(W_{yh}h^{<t>} + b_y)$$

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



The state consists of a single “hidden” vector h :

$$\mathbf{h}_t = f_W(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

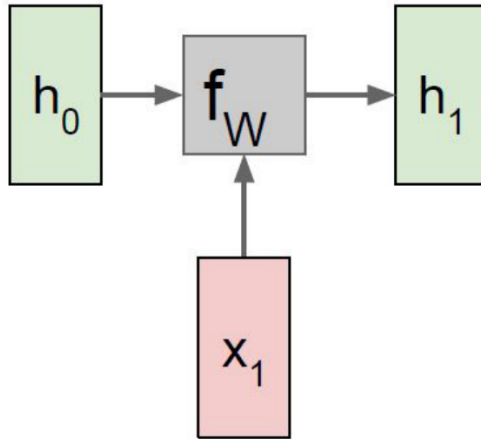
$$\mathbf{h}_t = \tanh(W_{hh}\mathbf{h}_{t-1} + W_{xh}\mathbf{x}_t)$$

$$\mathbf{y}_t = W_{hy}\mathbf{h}_t$$

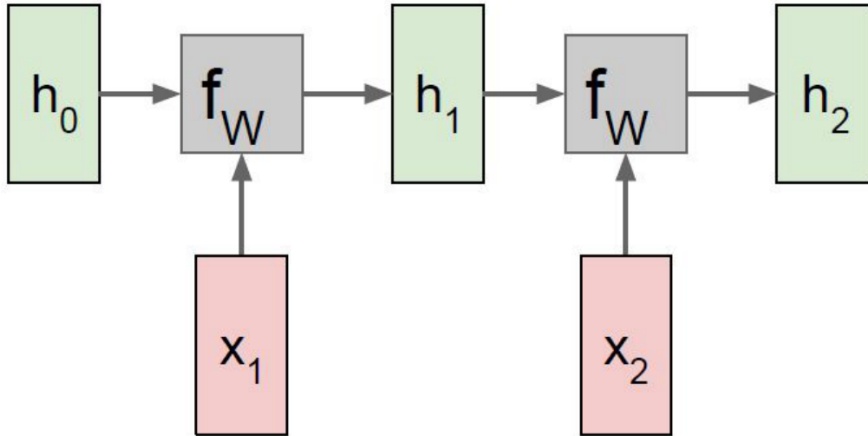
- Sometimes called a “Vanilla RNN” or an *Elman RNN* after Prof. Jeffrey Elman.

Adapted from: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks.

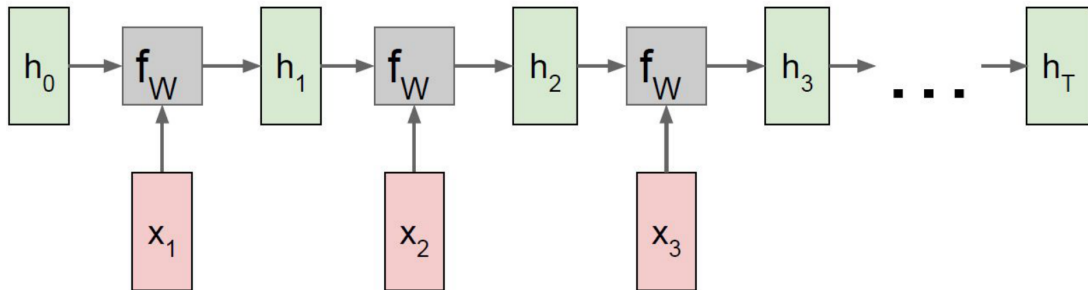
RNN : Parameter Sharing



Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

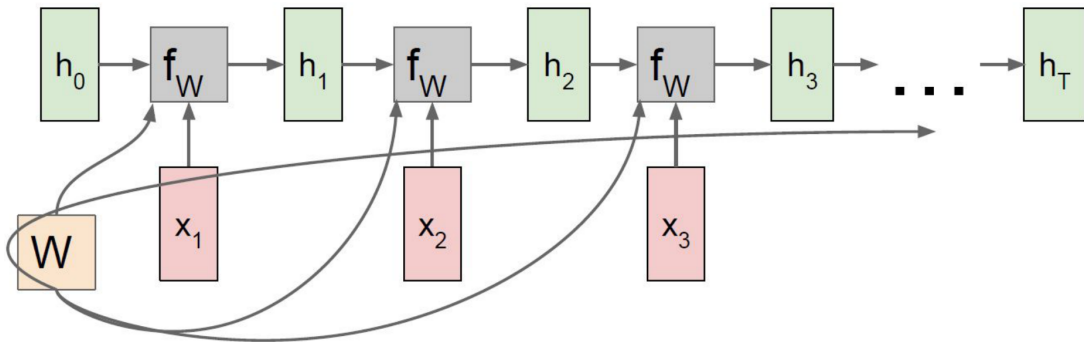


Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks



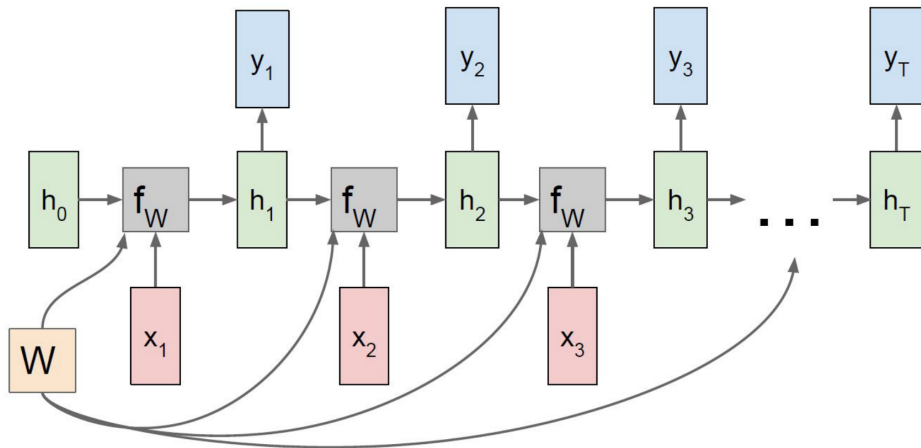
Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

Re-use the same weight matrix at every time-step



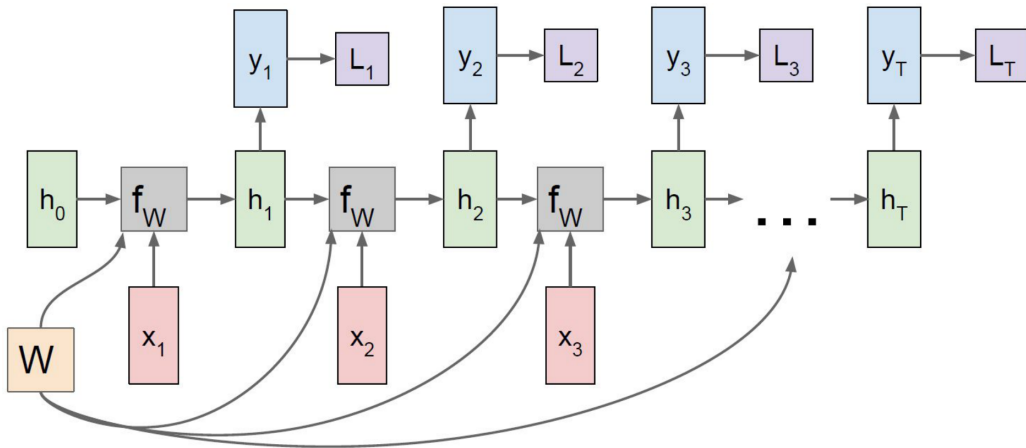
Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

RNN Computation Graph - Many to Many



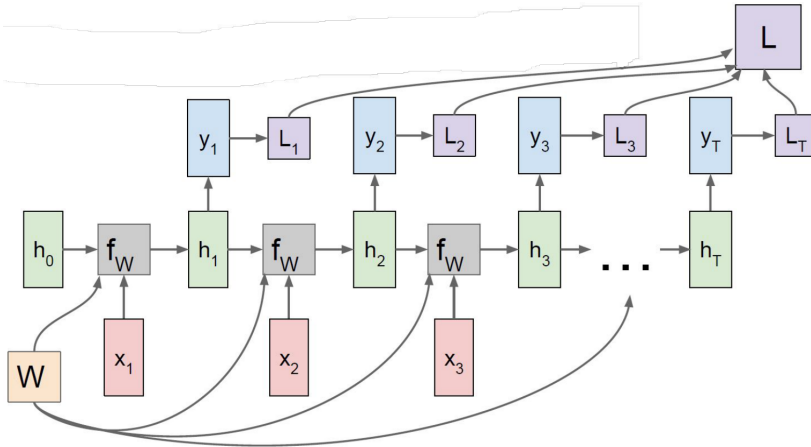
Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

RNN Computation Graph - Many to Many



Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

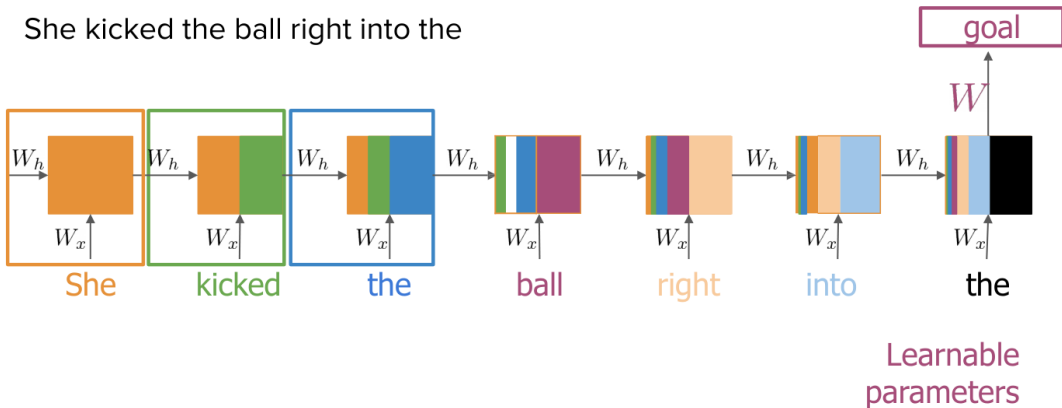
RNN Computation Graph - Many to Many



Source: Stanford CS231n (2022), Lecture 10: Recurrent Neural Networks

RNN : Challenges

She kicked the ball right into the



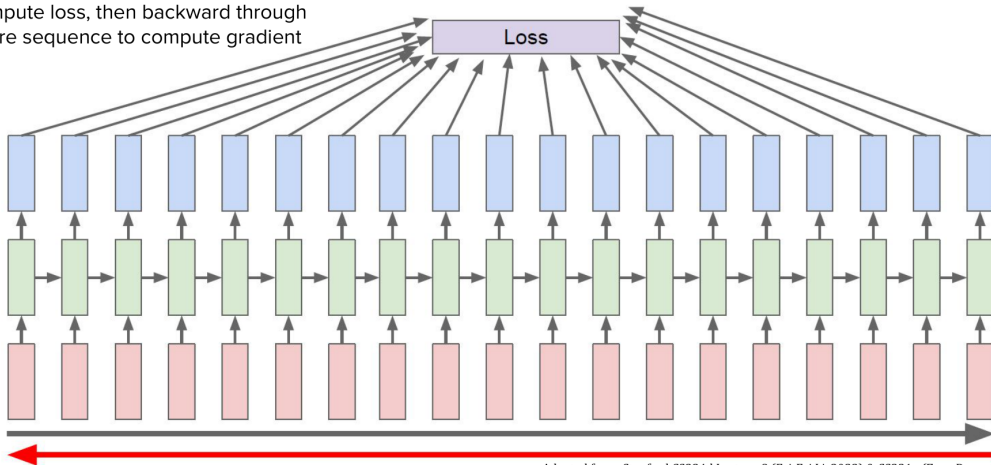
Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)

Backpropagation Through Time (BPTT)

- ▶ Training RNNs involves unfolding the network across time steps.
- ▶ Standard backpropagation is applied through this unrolled structure.

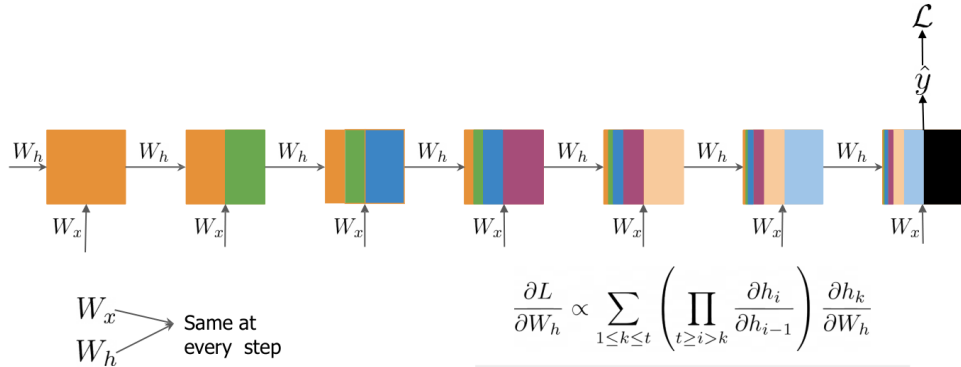
RNN: Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Backpropagation through time



$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \leq k \leq t} \left(\prod_{t \geq i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h}$$

Gradient is proportional to a sum of partial derivative products

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)

$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \leq k \leq t} \left(\prod_{t \geq i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h}$$

Contribution of hidden state k

Length of the product proportional to
how far k is from t

$$\frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial h_{t-2}} \frac{\partial h_{t-2}}{\partial h_{t-3}} \frac{\partial h_{t-3}}{\partial h_{t-4}} \frac{\partial h_{t-4}}{\partial h_{t-5}} \frac{\partial h_{t-5}}{\partial h_{t-6}} \frac{\partial h_{t-6}}{\partial h_{t-7}} \frac{\partial h_{t-7}}{\partial h_{t-8}} \frac{\partial h_{t-8}}{\partial h_{t-9}} \frac{\partial h_{t-9}}{\partial h_{t-10}} \frac{\partial h_{t-10}}{\partial W_h}$$

Contribution of hidden state $t-10$

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)

$$\frac{\partial L}{\partial W_h} \propto \sum_{1 \leq k \leq t} \left(\prod_{t \geq i > k} \frac{\partial h_i}{\partial h_{i-1}} \right) \frac{\partial h_k}{\partial W_h}$$

Contribution of hidden state k

Length of the product proportional to how far k is from t

Partial derivatives < 1	Contribution goes to 0	Vanishing Gradient
Partial derivatives > 1	Contribution goes to infinity	Exploding Gradient

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)

Problems

- ▶ **Vanishing Gradients:** Gradients shrink as they are propagated back, making it hard to learn long-term dependencies.
- ▶ **Exploding Gradients:** Gradients grow exponentially, leading to unstable updates.

Solutions (covered next)

- ▶ Use of LSTM and GRU architectures
- ▶ Gradient Clipping

The problem when training generative RNNs

- ▶ A generating RNN predicts token y_t from its own previous output $\hat{y}_{t-1}$.
- ▶ Early in training the model's outputs are mostly wrong, so errors feed back as inputs and compound across the sequence.

Teacher forcing

- ▶ During training, feed the ground-truth token y_{t-1} as the input at step t , regardless of what the model predicted.
- ▶ Every step is trained on a correct prefix, so learning is faster and more stable.
- ▶ At inference there is no ground truth: the model must consume its own predictions.

- ▶ **Exposure bias:** with teacher forcing, the model is trained only on ground-truth prefixes but tested on its own imperfect prefixes. One early mistake at inference can push it into states it never saw during training.
- ▶ **Scheduled sampling** (Bengio et al., 2015): during training, feed the model its own prediction instead of the ground truth with some probability, and increase that probability over the course of training.
- ▶ The model gradually learns to recover from its own mistakes while keeping the stability of teacher forcing early on.

Backpropagation Through Time (BPTT) spreads gradients across many time steps.

Vanishing Gradients:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\| \rightarrow 0$$

- ▶ Early layers barely learn
- ▶ Forget long-term dependencies

Exploding Gradients:

$$\left\| \frac{\partial \mathcal{L}}{\partial h_t} \right\| \rightarrow \infty$$

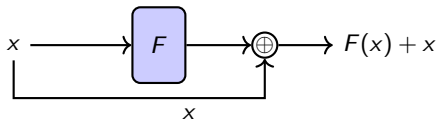
- ▶ Unstable updates, diverging weights

► Identity RNN with ReLU activation

► Gradient clipping

► Skip connections

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$



Exploding gradients are a problem of *magnitude*, so they have a cheap fix: after backpropagation, rescale the gradient whenever it grows past a threshold τ .

$$g \leftarrow \frac{\partial \mathcal{L}}{\partial \theta}, \quad \text{if } \|g\| > \tau \quad \text{then} \quad g \leftarrow \frac{\tau}{\|g\|} g$$

- ▶ The rescaling uses the norm of the gradient over *all* parameters at once, so the update *direction* is preserved and only its length is capped.
- ▶ τ is a hyperparameter. Pascanu et al. (2013) suggest setting it from the average gradient norm observed over a stable stretch of training.
- ▶ It stops one bad batch from destroying the weights, but it does *not* address vanishing gradients: rescaling a gradient that has already decayed recovers no information about distant time steps. That is what gating is for.

RNN Advantages

- ▶ Can process any length input.
- ▶ Computation for step t can (in theory) use information from many steps back.
- ▶ Model size doesn't increase for longer input.
- ▶ Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages

- ▶ Recurrent computation is slow
- ▶ In practice, difficult to access information from many steps back

LSTM : Long Short-Term Memory

Introduced by Hochreiter & Schmidhuber (1997)

Core Idea: LSTM uses **gates** to control what to keep, forget, and output.

Key Components:

- ▶ **Forget Gate** f_t
- ▶ **Input Gate** i_t
- ▶ **Cell State** C_t
- ▶ **Output Gate** o_t

The 1997 architecture had only the input and output gates. The forget gate was added by Gers, Schmidhuber & Cummins (2000), and the three-gate unit below is what “LSTM” now means.

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

Hidden state

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

Equations:

$$f_t = \sigma (W_f[x_t, h_{t-1}] + b_f)$$

$$i_t = \sigma (W_i[x_t, h_{t-1}] + b_i)$$

$$\tilde{C}_t = \tanh (W_c[x_t, h_{t-1}] + b_c)$$

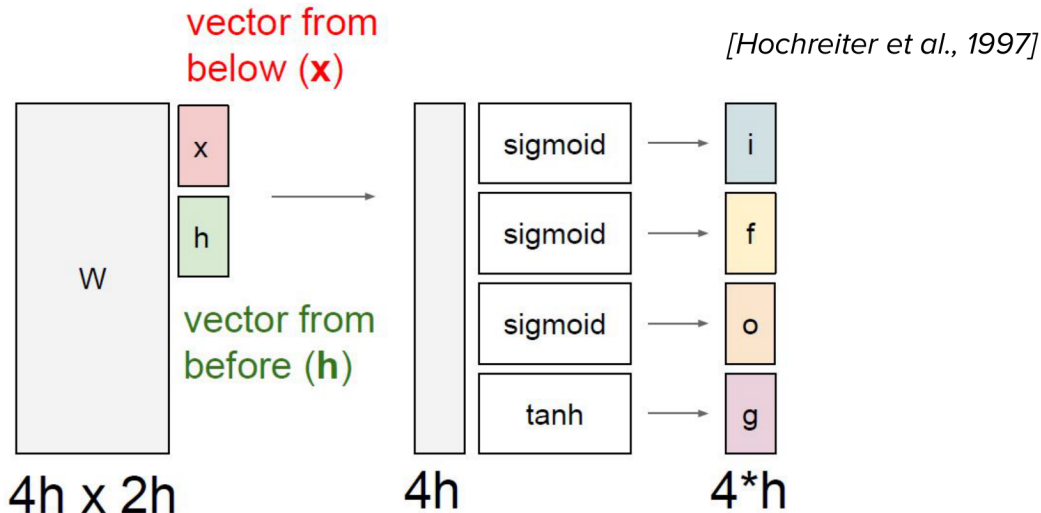
$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

$$o_t = \sigma (W_o[x_t, h_{t-1}] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

Helps retain long-term dependencies

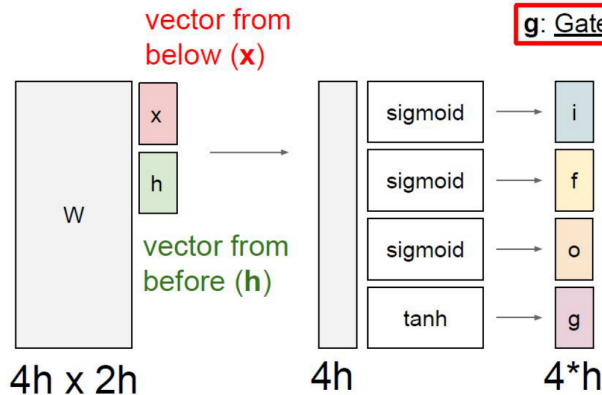
- ▶ Learns when to remember and when to forget
- ▶ **Basic anatomy:**
 - A cell state
 - A hidden state
 - Multiple gates
- ▶ Gates allow gradients to avoid vanishing and exploding



Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

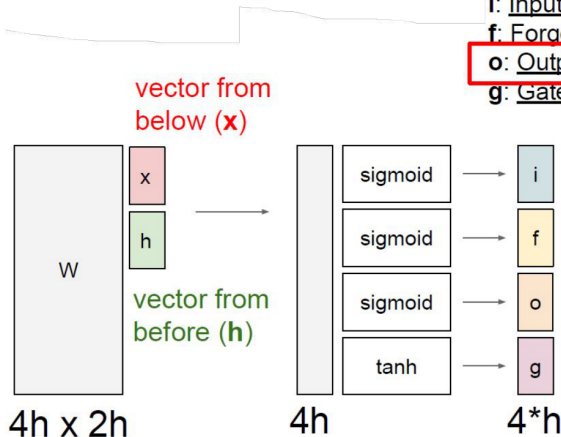
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

- i: Input gate, whether to write to cell
- f: Forget gate. Whether to erase cell
- o: Output gate, How much to reveal cell**
- g: Gate gate (?), How much to write to cell



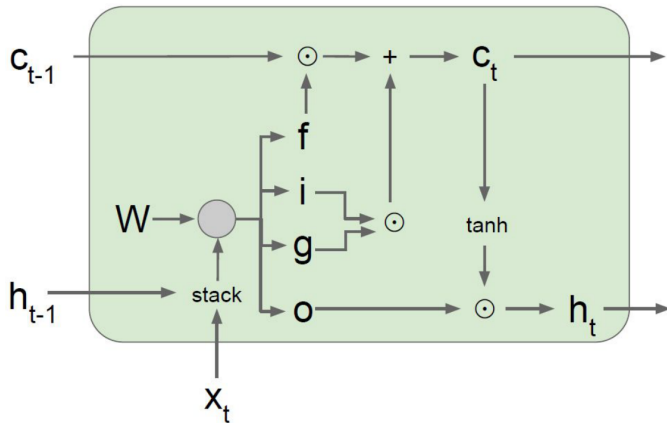
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = \boxed{o} \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

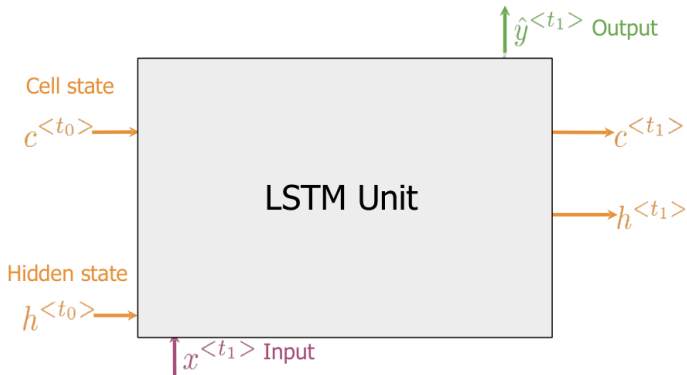


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

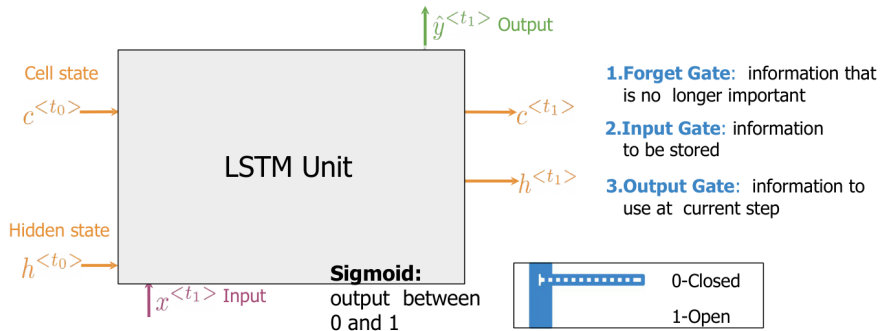
$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

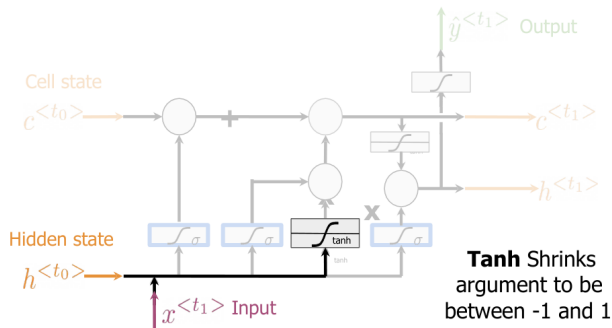


- 1. Forget Gate:** information that is no longer important
- 2. Input Gate:** information to be stored
- 3. Output Gate:** information to use at current step

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)



Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)

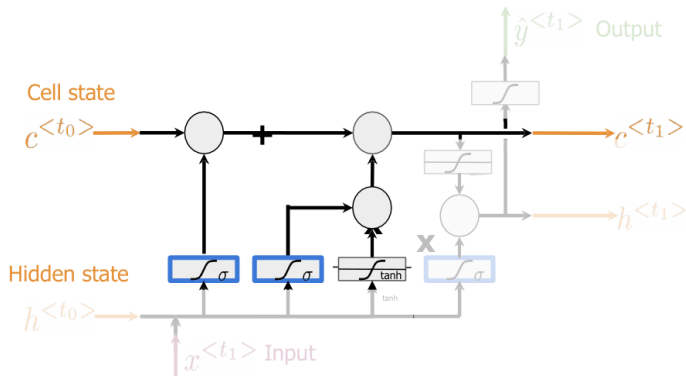


Candidate cell state

Information from the previous **hidden state** and current **input**

Other activations could be used

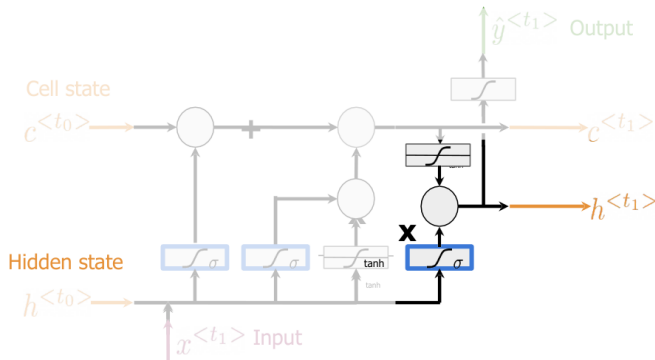
Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)



New Cell state

Add information from the **candidate cell state** using the **forget** and **input gates**

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)



New Hidden State

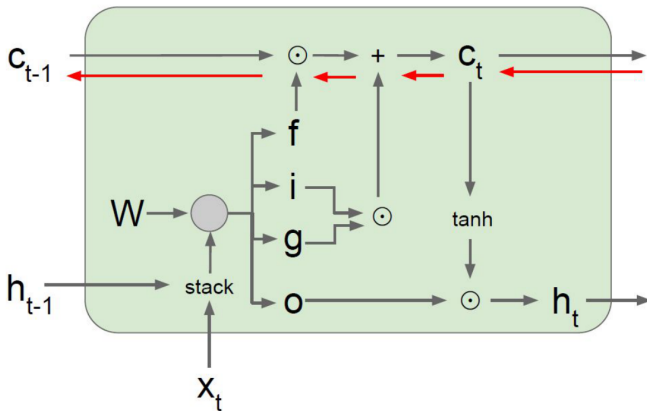
Select information from the **new cell state** using the **output gate**

The **Tanh** activation could be omitted

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 3 (Y. Bensouda Mourri & L. Kaiser)

[Hochreiter et al., 1997]

Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

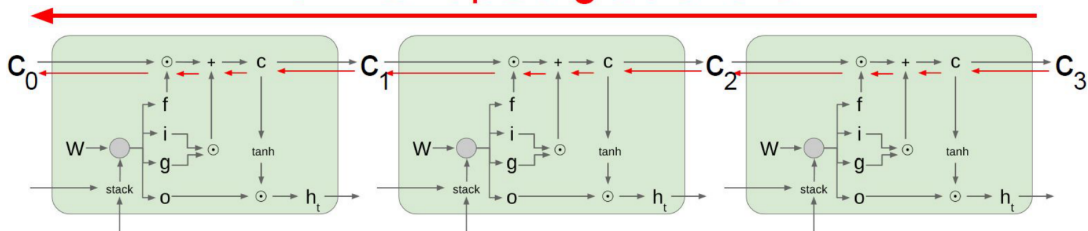
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

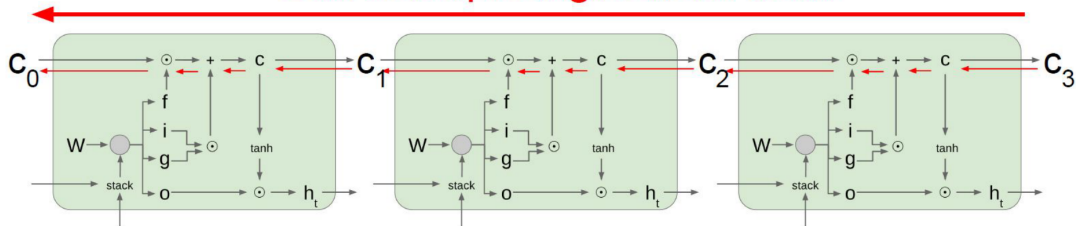
Uninterrupted gradient flow!



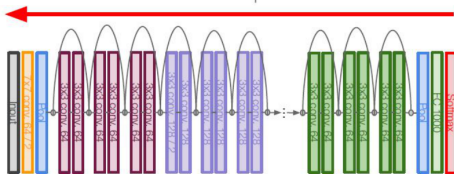
Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

[Hochreiter et al., 1997]

Uninterrupted gradient flow!



Similar to ResNet!



In between:
Highway Networks

$$g = T(x, W_T)$$

$$y = g \odot H(x, W_H) + (1 - g) \odot x$$

Srivastava et al, "Highway Networks",
ICML DL Workshop 2015

Adapted from: Stanford CS224d Lecture-8 (Fei-Fei Li, 2023) & CS231n (Zane Durante, 2025)

Do LSTMs solve the vanishing gradient problem?

- ▶ The LSTM architecture makes it easier for the RNN to preserve information over many timesteps.
 - e.g. if the forget gate $f = 1$ and input gate $i = 0$, then the information of that cell is preserved indefinitely.
 - By contrast, it is harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves information in the hidden state.
- ▶ LSTM **doesn't guarantee** that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies.

GRU : Gated Recurrent Unit

Introduced by Cho et al. (2014)

Core Idea: GRU merges the cell state into the hidden state and uses two gates to control how much of the past to keep and how much of the new candidate to write.

Key Components:

- ▶ **Update Gate z_t**
- ▶ **Reset Gate r_t**

Equations:

$$z_t = \sigma(W_z[x_t, h_{t-1}] + b_z)$$

$$r_t = \sigma(W_r[x_t, h_{t-1}] + b_r)$$

$$\tilde{h}_t = \tanh(W_h[x_t, r_t * h_{t-1}] + b_h)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

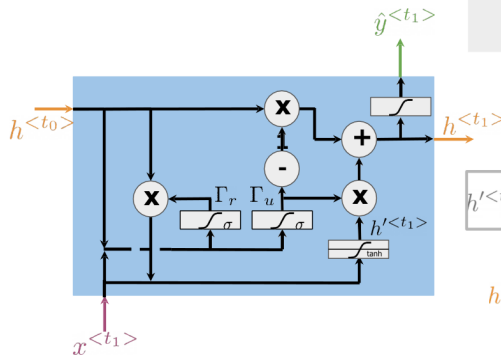
Comparable performance to LSTM
Often faster to train, fewer parameters

"Ants are really interesting. They are everywhere."

↓
Plural

Relevance and update gates to remember important prior information

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



Gates to keep/update relevant information in the hidden state

$$\begin{aligned}\Gamma_r &= \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r) \\ \Gamma_u &= \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)\end{aligned}$$

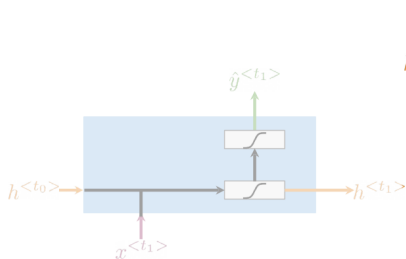
$$h'^{<t_1>} = \tanh(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h)$$

Hidden state candidate

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

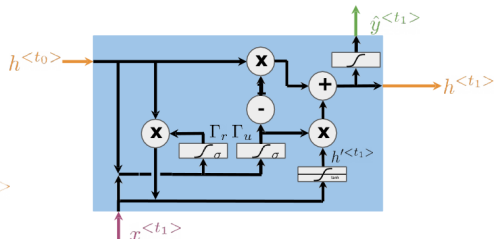
$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)



$$h^{<t>} = g(W_h[h^{<t-1>}, x^{<t>}] + b_h)$$

$$\hat{y}^{<t>} = g(W_y h^{<t>} + b_y)$$



$$\Gamma_u = \sigma(W_u[h^{<t_0>}, x^{<t_1>}] + b_u)$$

$$\Gamma_r = \sigma(W_r[h^{<t_0>}, x^{<t_1>}] + b_r)$$

$$h'^{<t_1>} = \tanh(W_h[\Gamma_r * h^{<t_0>}, x^{<t_1>}] + b_h)$$

$$h^{<t_1>} = (1 - \Gamma_u) * h^{<t_0>} + \Gamma_u * h'^{<t_1>}$$

$$\hat{y}^{<t_1>} = g(W_y h^{<t_1>} + b_y)$$

Source: DeepLearning.AI NLP Specialization, Course 3: Natural Language Processing with Sequence Models, Week 2 (Y. Bensouda Mourri & L. Kaiser)

Feature	LSTM	GRU
Gates	3 (i, f, o)	2 (z, r)
Memory Cell	Yes	No
Complexity	Higher	Lower
Training Speed	Slower	Often faster
Performance	Comparable; which one wins depends on the task	

Chung et al. (2014) compared the two units under matched parameter counts and could not declare a winner: the better choice varied with the dataset and the task.

Tip: Try GRU first for faster results; switch to LSTM if performance suffers.

NLP Tasks:

- ▶ Language Modeling
- ▶ Machine Translation
- ▶ Sentiment Analysis
- ▶ Chatbots

Audio & Time Series:

- ▶ Music Generation
- ▶ Speech Recognition
- ▶ Anomaly Detection

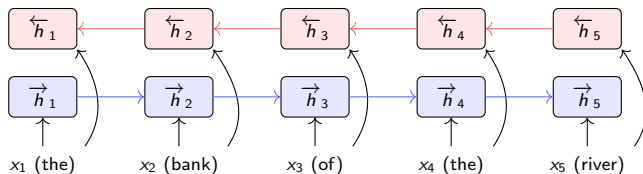
Video & Sequential Vision:

- ▶ Action Recognition
- ▶ Video Captioning

- ▶ Sequential computation: hard to parallelize across time steps
- ▶ Gating mitigates but does not remove the problem; very long dependencies are still lost
- ▶ Slow training due to sequential nature
- ▶ Batching variable-length sequences requires padding and masking

RNN : **Bidirectional RNNs**

- ▶ A standard RNN reads left to right: h_t summarizes only the past $x_1, \dots, x_t$.
- ▶ Many tasks need context from both sides. In “The bank of the river”, the meaning of “bank” depends on words that come later.
- ▶ A **bidirectional RNN** (Schuster and Paliwal, 1997) runs two RNNs over the sequence: a forward RNN producing $\vec{h}_t$ and a backward RNN producing $\overleftarrow{h}_t$.
- ▶ The representation of each position is the concatenation $h_t = [\vec{h}_t; \overleftarrow{h}_t]$: past and future context at every step.



- ▶ **Works for encoding:** the full input is available before any output is produced.
 - Part-of-speech tagging, named entity recognition, sequence classification.
 - BiLSTMs were the standard sentence encoder before transformers.
- ▶ **Does not work for generation:** when producing text token by token, the future does not exist yet, so the backward RNN has nothing to read. Decoders stay unidirectional.
- ▶ This encoder/decoder split carries into modern models: bidirectional encoders and left-to-right decoders.

- ▶ **Transformers:** Fully parallelized sequence modeling using attention
- ▶ **Efficient Attention:** Longformer, Linformer, etc. for long sequences
- ▶ **Neural Memory Networks:** Explicit memory read/write
- ▶ **Recurrent Attention Models**
- ▶ **Hybrid Architectures:** RNN + CNN + Attention

RNNs are still used in edge devices for efficient modeling

- ▶ RNNs struggle with long dependencies due to vanishing/exploding gradients
- ▶ GRU and LSTM improve memory retention using gating mechanisms
- ▶ GRU is simpler and faster; LSTM is more expressive
- ▶ Attention and transformers now dominate, but RNNs remain relevant in many domains

These slides have been adapted from

- ▶ Younes Mourri & Lukasz Kaiser, [Natural Language Processing Specialization, DeepLearning.AI](#)
- ▶ Fei-Fei Li, Jiajun Wu & Ruohan Gao, [Stanford CS231n \(2022\), Lecture 10: Recurrent Neural Networks](#)
- ▶ Andrej Karpathy, [The Unreasonable Effectiveness of Recurrent Neural Networks \(2015\)](#)

Foundational Papers:

- ▶ Elman, J. L. (1990). *Finding Structure in Time*. Cognitive Science, 14(2).
- ▶ Bengio, Y., Simard, P., & Frasconi, P. (1994). *Learning long-term dependencies with gradient descent is difficult*. IEEE Transactions on Neural Networks.
- ▶ Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation.
- ▶ Schuster, M., & Paliwal, K. K. (1997). *Bidirectional Recurrent Neural Networks*. IEEE Transactions on Signal Processing.
- ▶ Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). *Learning to Forget: Continual Prediction with LSTM*. Neural Computation.

Foundational Papers (continued):

- ▶ Pascanu, R., Mikolov, T., & Bengio, Y. (2013). *On the difficulty of training recurrent neural networks*. ICML.
- ▶ Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP.
- ▶ Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. arXiv:1412.3555.
- ▶ Le, Q. V., Jaitly, N., & Hinton, G. E. (2015). *A Simple Way to Initialize Recurrent Networks of Rectified Linear Units*. arXiv:1504.00941.
- ▶ Bengio, S., Vinyals, O., Jaitly, N., & Shazeer, N. (2015). *Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks*. NeurIPS.
- ▶ Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS.

Resources:

- ▶ Karpathy's RNN Blog:
<https://karpathy.github.io/2015/05/21/rnn-effectiveness/>
- ▶ CS231n Lecture Notes on RNNs and LSTM
- ▶ DeepLearning.AI NLP Specialization (Coursera)
- ▶ MIT 6.S191 Deep Learning Lecture Slides